
Credit Card Fraud Detection Using Machine Learning

Project Id: AIML-12

Name : Riddhima Sharma(Team Leader)

Class Roll Number: 21

University Roll Number : 2028474

Section and Group: U(G1)

Name : Disha Tahiliani

Class Roll Number: 11

University Roll Number : 2028413

Section and Group: U(G1)

Name : Aditya Rawal

Class Roll Number: 5

University Roll Number : 2028383

Section and Group: U(G1)

TABLE OF CONTENTS

1. Problem Description	3
1.1.Dataset Description.....	3
1.2.Flow diagram.....	4
2. Modules for Project Implementation.....	5
2.1. Modules Used	 5
2.2. Platform Used	 6
3. Machine Learning model/ Searching Technique used	7
4. Evaluation metrics used	8
5. Results and Visualizations	9
6. Conclusion and Future Scope	11
7. References.....	12

1. Problem Description

Credit card fraud detection is a critical issue in financial systems due to the increasing volume of digital transactions. Manual detection methods are inefficient and prone to errors.

This project develops a machine learning-based fraud detection system that classifies transactions as:

Legitimate (Class 0)

Fraudulent (Class 1)

The system focuses on improving fraud detection accuracy while minimizing false alarms.

1.1 Dataset Description

The dataset used is the Credit Card Fraud Detection Dataset (Kaggle).

Dataset Features:

Total transactions: 284,807

Fraud cases: ~0.17%

Features:

V1–V28 (PCA transformed features)

Time

Amount

Class (target variable)

Preprocessing Steps:

Standardization of features

Log transformation of Amount

Handling missing values

SMOTE applied for balancing

1.2 Flow Diagram

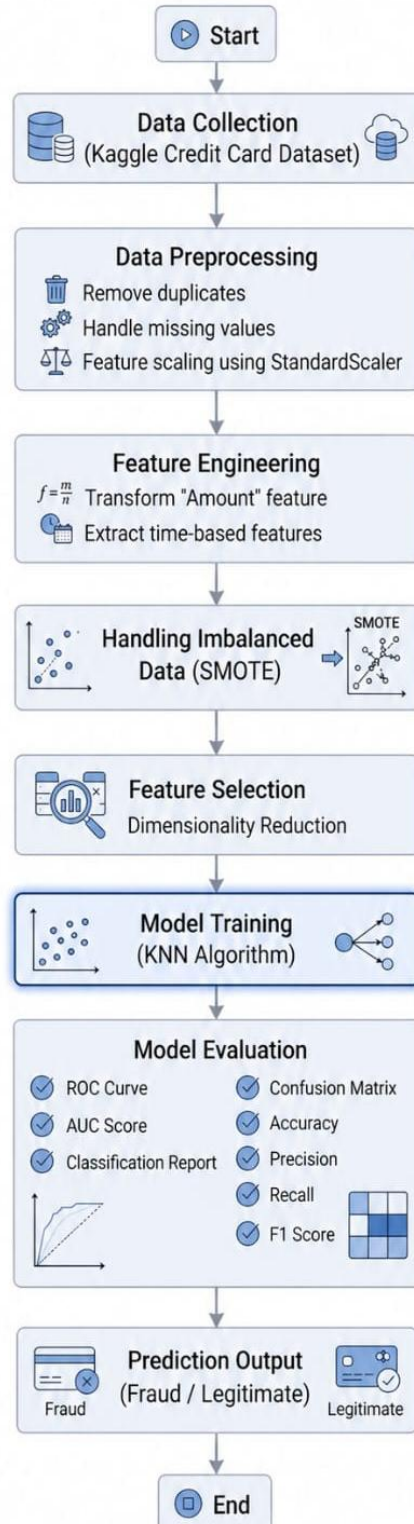


Fig.1 Sample flow diagram.

2. Modules for Project Implementation

2.1 Modules Used(mention all the inbuilt/ custom modules used and the purpose)

Modules Used

NumPy

Used for numerical operations and array handling

Pandas

Data loading and manipulation

Functions:

read_csv()

drop()

describe()

Matplotlib / Seaborn

Used for visualization

Heatmaps, plots, graphs

Scikit-learn

Machine learning implementation

Functions:

train_test_split()

StandardScaler()

KNeighborsClassifier()

accuracy_score()

confusion_matrix()

Imbalanced-learn (SMOTE)

Used to balance dataset

Prevents bias toward majority class

2.2 Platform Used: (mention all the platforms with the reason of using it)

- Python 3.10+
- Jupyter Notebook (for development & visualization)
- Visual Studio Code (for coding/debugging)
- Windows OS

3. Machine Learning Model/ Searching Technique Used (Describe each ML model/ searching technique used)

K-Nearest Neighbors (KNN)

KNN is used as the primary classification model.

Working Process:

Choose value of K

Compute distance between data points

Identify nearest neighbors

Assign class using majority voting

Distance Formula (Euclidean Distance):

$$d = \sum [(x_i - y_i)^2]^{0.5}$$

Advantages:

Simple and effective

No training required

Works well with small datasets

Limitations:

Slow for large datasets

Sensitive to noise

Requires feature scaling

4. Evaluation metrics used (Describe each evaluation metric used along with formulae)

Accuracy

Accuracy =

$\frac{TP + TN}{TP + TN + FP + FN}$

Precision
 $\text{Precision} = \frac{TP}{TP + FP}$

Recall
 $\text{Recall} = \frac{TP}{TP + FN}$

F1 Score
 $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Confusion Matrix

Used to analyze predictions:

True Positive
False Positive
True Negative
False Negative
ROC Curve & AUC
Evaluates model performance
AUC close to 1 indicates strong model

5. Results and Visualizations

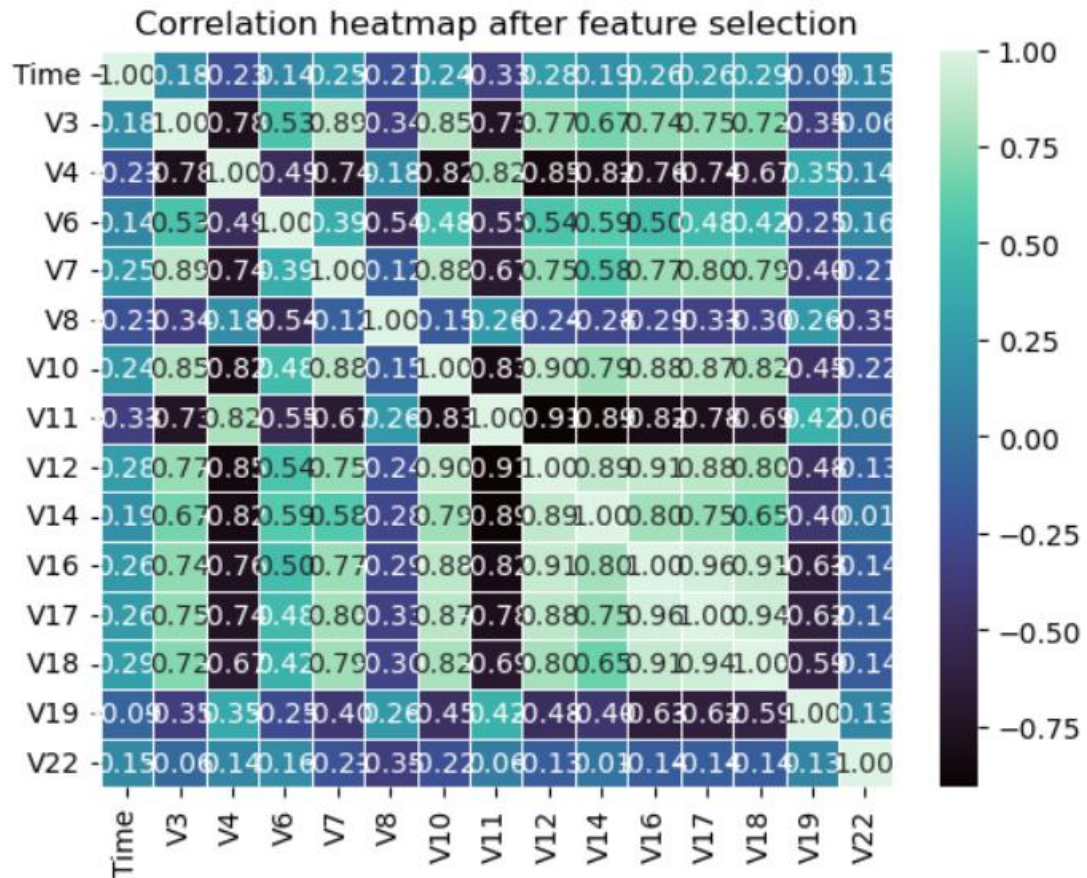


Fig.2 Heatmap showing correlation.

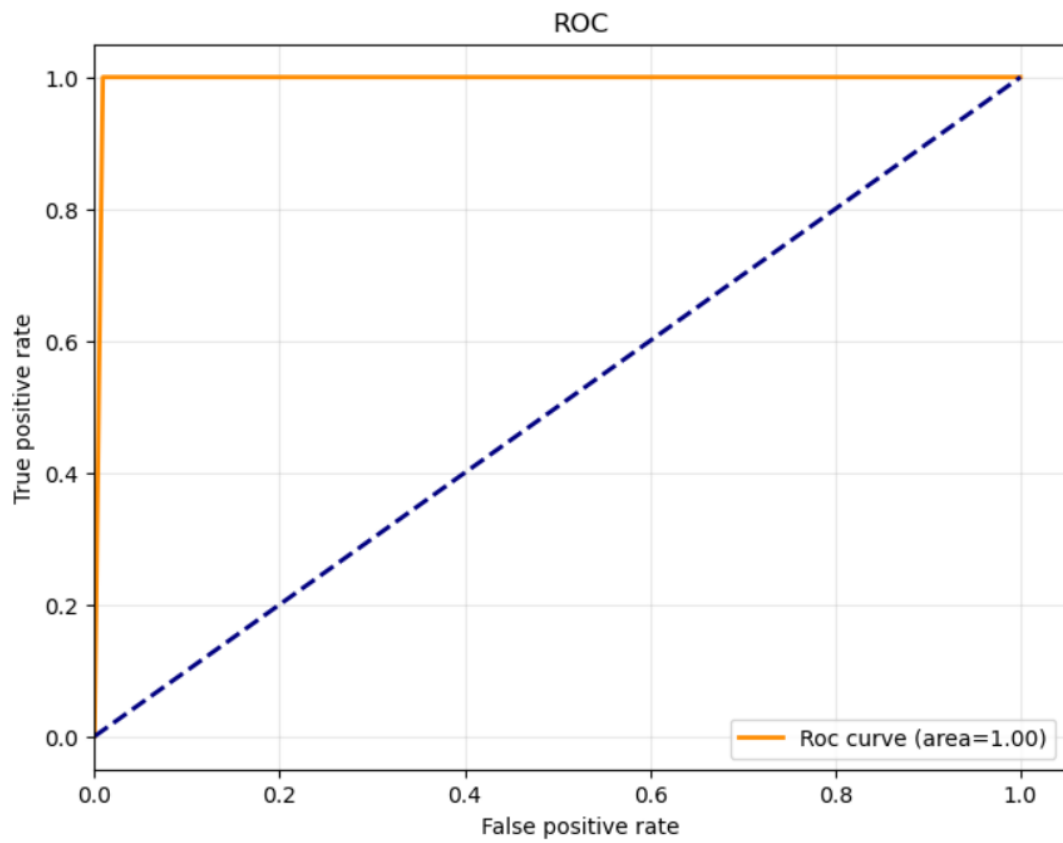


Fig.3 roc curve of knn

	precision	recall	f1-score	support
0	1.00	0.99	1.00	56596
1	0.99	1.00	1.00	55446
accuracy			1.00	112042
macro avg	1.00	1.00	1.00	112042
weighted avg	1.00	1.00	1.00	112042

Fig.4 knn

```
Model loaded successfully
['Time', 'V3', 'V4', 'V6', 'V7', 'V8', 'V10', 'V11', 'V
12', 'V14', 'V16', 'V17', 'V18', 'V19', 'V22']
Enter above mentioned features: 0 2.536346738 1.3781552
24 0.462387778 0.239598554 0.098697901 0.090794172 -0.5
51599533 -0.617800856 -0.311169354 -0.470400525 0.2079
71242 0.02579058 0.40399296 0.277837576
0:Legitimate Transaction
```

Fig 4.1

```
Model loaded successfully
['Time', 'V3', 'V4', 'V6', 'V7', 'V8', 'V10', 'V11', 'V
12', 'V14', 'V16', 'V17', 'V18', 'V19', 'V22']
Enter above mentioned features: 406 -1.609850732 3.9979
05588 -1.426545319 -2.537387306 1.391657248 -2.77227214
5 3.202033207 -2.899907388 -4.289253782 -1.14074718 -2.
830055675 -0.016822468 0.416955705 -0.035049369
1:Fraudulent Transaction
```

Fig 4.2

6. Conclusion and Future Scope:

Conclusion

The project Credit Card Fraud Detection Using Machine Learning successfully demonstrates how data-driven approaches can be used to tackle real-world financial security problems. With the rapid growth of online transactions, detecting fraudulent activities has become increasingly complex, and traditional rule-based systems are no longer sufficient.

In this project, a machine learning model based on the K-Nearest Neighbors (KNN) algorithm was implemented to classify transactions as legitimate or fraudulent. The dataset used was highly imbalanced, which is a common challenge in fraud detection systems. This issue was effectively handled using SMOTE (Synthetic Minority Oversampling Technique), which improved the model's ability to detect minority class instances (fraud cases).

Proper preprocessing techniques such as feature scaling, transformation, and data cleaning played a crucial role in enhancing model performance. The use of evaluation metrics like accuracy, precision, recall, F1-score, and ROC-AUC ensured a comprehensive assessment of the model, especially in identifying fraud cases where recall is critically important.

The results indicate that the model is capable of detecting fraudulent transactions with reasonable accuracy while maintaining a balance between false positives and false negatives. Visualizations such as heatmaps and ROC curves further helped in understanding feature relationships and model performance.

Overall, this project highlights the importance of:

- Handling imbalanced datasets
- Choosing appropriate evaluation metrics
- Applying suitable machine learning techniques

It provides a solid foundation for building more advanced and scalable fraud detection systems.

Future Scope

While the current system performs effectively, there are several areas where it can be further improved and expanded:

1. Implementation of Advanced Models

Future work can include more powerful algorithms such as:

- Random Forest
- XGBoost
- Logistic Regression
- Deep Learning models (ANN, LSTM)

These models can improve accuracy, handle large datasets better, and reduce overfitting.

2. Real-Time Fraud Detection System

The current model works on historical data. It can be extended to:

- Real-time transaction monitoring
- Instant fraud alerts
- Integration with live banking systems

This would make the system more practical and industry-ready.

3. Feature Engineering and Data Enhancement

More meaningful features can be created, such as:

- Transaction frequency patterns
- User behavior analysis
- Geolocation-based anomalies

This can significantly improve prediction performance.

4. Handling Large-Scale Data

To improve scalability:

- Use distributed computing frameworks (like Spark)
- Optimize algorithms for faster prediction
- Implement cloud-based deployment

5. Reduction of False Positives

In real-world systems, false alarms can inconvenience users. Future improvements can focus on:

- Better threshold tuning
- Ensemble techniques
- Cost-sensitive learning

6. Integration with Banking APIs

The system can be connected to:

- Banking applications
- Payment gateways
- Financial dashboards

This will enable automated fraud prevention systems.

7. Explainable AI (XAI)

Adding interpretability to the model will help:

- Understand why a transaction is flagged as fraud
- Build trust among users and financial institutions

7. References:

Books:

1. Let Us Python – Yashavant Kanetkar

Online:

- [Kaggle Dataset](#)
- [Scikit-learn Documentation](#)
- [imbalanced-learn Documentation](#)

